

Simplified MPEG-4 Video Encoder Pipeline

Multimedia Systems — Mini Project Report

GitHub: github.com/wailman24/multiproject

May 2026

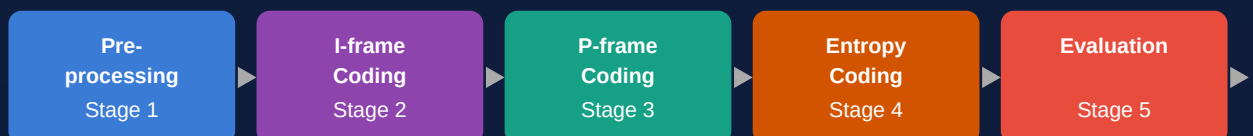


Table of Contents

1.	Project Overview & Pipeline Architecture	3
2.	Repository Structure	3
3.	Part 1 — Pre-processing (Color Space + Chroma Subsampling)	4
4.	Part 2 — Intra-frame Coding (I-frames)	5
5.	Part 3 — Inter-frame Coding (P-frames)	7
6.	Part 4 — Entropy Coding	9
7.	Part 5 — Evaluation & Visualisation	10
8.	Design Choices & Justification	11
9.	Experimental Analysis	12
10.	Conclusion	13

1. Project Overview & Pipeline Architecture

This project implements a complete simplified MPEG-4-like video codec in Python. Given a folder of sequential PNG/JPG frames representing a short video clip, the encoder compresses them into a single binary **.bin** file. The decoder reconstructs the original frames from this file. The pipeline closely mirrors the structure of real-world video codecs such as H.264 or MPEG-4.

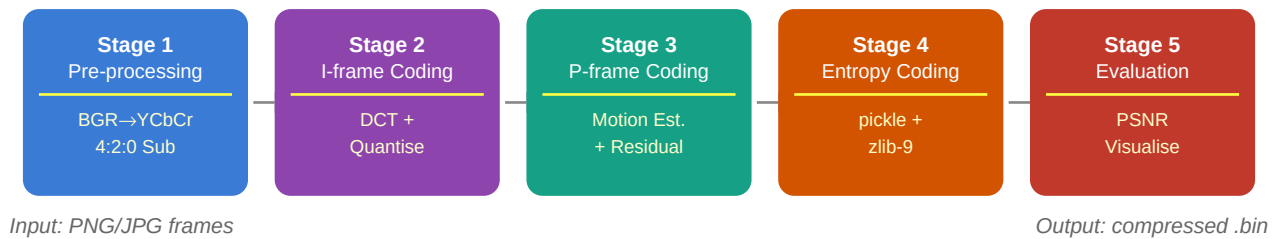


Figure 1 — The five-stage encoder pipeline from raw frames to compressed binary.

Key Concepts

A video codec exploits two fundamental types of redundancy:

Spatial redundancy	Adjacent pixels in the same frame tend to have similar values → removed by DCT + quantisation (I-frames)
Temporal redundancy	Consecutive frames are nearly identical → removed by motion estimation (P-frames)
Statistical redundancy	Many coefficients are zero after quantisation → removed by entropy coding (zlib)

2. Repository Structure

The project is split into focused modules, one per pipeline stage:

```
multiproject/  
  ■■■ preprocessing.py # Part 1 — BGR↔YCbCr + 4:2:0 sub/upsample  
  ■■■ intra_coding.py # Part 2 — I-frame DCT + quantisation  
  ■■■ inter_coding.py # Part 3 — P-frame motion est. + residual  
  ■■■ entropy_coding.py # Part 4 — pickle + zlib → .bin  
  ■■■ encoder.py # Orchestrates Stages 1–4 (Encoder class)  
  ■■■ decoder.py # Reverses all stages (Decoder class)  
  ■■■ evaluation.py # Part 5 — metrics + matplotlib visualisation  
  ■■■ main.py # CLI: encode / decode / visualise / analyse  
  ■■■ generate_test_frames.py # Helper: synthetic moving-circle frames  
  ■■■ requirements.txt # numpy, opencv-python, scipy, matplotlib
```

The **encoder.py** and **decoder.py** classes are the top-level orchestrators. Each stage module is self-contained and can be tested independently. The **main.py** entry point exposes four CLI sub-commands: *encode*, *decode*, *visualise*, and *analyse*.

3. Part 1 — Pre-processing

Color Space Conversion: BGR → YCbCr

OpenCV loads images in BGR format. Before compression, each frame is converted to the **YCbCr color space** (BT.601 standard). This separates the brightness (luma, Y) from the color information (chroma, Cb and Cr). The human visual system is far more sensitive to brightness detail than to color detail — exploiting this is the first major compression opportunity.

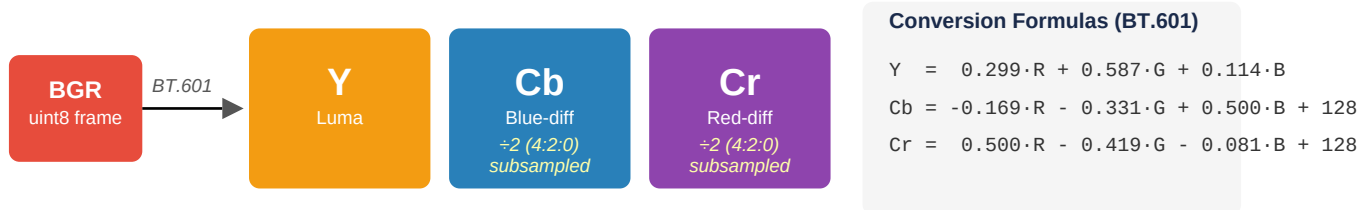


Figure 2 — BGR to YCbCr conversion and 4:2:0 chroma subsampling.

4:2:0 Chroma Subsampling

After conversion, the Cb and Cr channels are downsampled to half resolution in both dimensions by keeping every other sample. For a 320×240 frame, this gives:

Channel	Resolution	Data size
Y (luma)	320×240 (full)	76,800 values
Cb (blue-diff)	160×120 (quarter area)	19,200 values
Cr (red-diff)	160×120 (quarter area)	19,200 values
Total YCbCr	—	115,200 values (vs 230,400 for full BGR)

Key Code — preprocessing.py

```
def bgr_to_ycbcr(frame_bgr: np.ndarray):
    rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB).astype(np.float32)
    R, G, B = rgb[..., 0], rgb[..., 1], rgb[..., 2]
    Y = 0.299 * R + 0.587 * G + 0.114 * B
    Cb = -0.168736 * R - 0.331264 * G + 0.5 * B + 128.0
    Cr = 0.5 * R - 0.418688 * G - 0.081312 * B + 128.0
    return Y, Cb, Cr

def chroma_subsample_420(Cb, Cr):
    # Keep every other sample in both axes → half resolution
    return Cb[::2, ::2].copy(), Cr[::2, ::2].copy()

def chroma_upsample_420(Cb_sub, Cr_sub, target_h, target_w):
    # Bilinear interpolation back to full resolution (decoder)
    Cb_up = cv2.resize(Cb_sub, (target_w, target_h), interpolation=cv2.INTER_LINEAR)
    Cr_up = cv2.resize(Cr_sub, (target_w, target_h), interpolation=cv2.INTER_LINEAR)
```

```
return Cb_up, Cr_up
```

Code Listing 1 — preprocessing.py core functions.

4. Part 2 — Intra-frame Coding (I-frames)

Every G -th frame (frame 0, G , $2G$, ...) is coded as an **I-frame** — compressed entirely on its own without reference to any other frame. This is essentially JPEG compression applied to video frames.

I-frame DCT Encoding Pipeline (per 8×8 block, per channel)

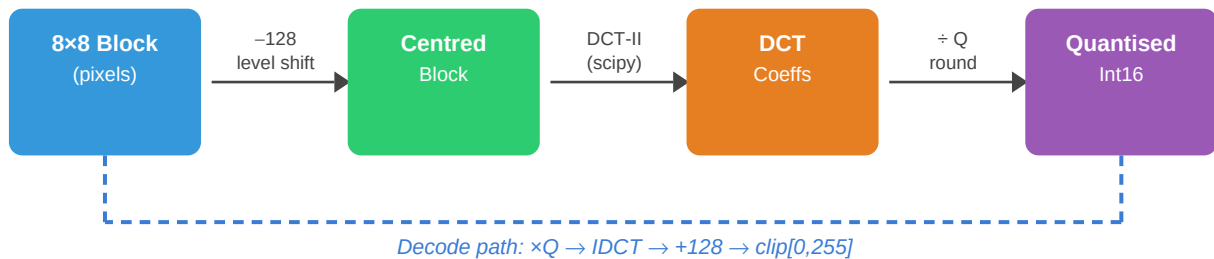


Figure 3 — The I-frame DCT encoding and decoding pipeline for each 8×8 block.

Step-by-Step Process

- 1. Pad:** Edge-pad each channel so both dimensions are multiples of 8.
- 2. Tile:** Split the padded channel into 8×8 non-overlapping blocks.
- 3. Level-shift:** Subtract 128 from each block to centre values around zero (required by DCT).
- 4. 2D DCT:** Apply `scipy.fft.dctn` with `norm="ortho"`. Energy concentrates in the top-left (DC + low-freq AC).
- 5. Quantise:** Divide each coefficient by the JPEG quantisation table entry and round. High-freq coeffs become 0.
- 6. Store:** Save as int16 array (dtype). The many zeros compress well in Stage 4.

JPEG Quantisation Table

The table below is the standard JPEG luminance quantisation matrix. Small values in the top-left preserve low frequencies (important for visual quality); large values in the bottom-right aggressively discard high frequencies. The **QF (quantisation factor)** multiplies every entry — higher QF = more compression = lower quality.

JPEG Luminance Quantisation Table (Q)

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Low values (top-left):
preserve DC & low-freq → fine detail kept

High values (bottom-right):
discard high-freq AC → most become 0

QF multiplier: $Q_scaled = clip(Q_base \times QF, 1, 255)$
 $quantised[i,j] = round(DCT[i,j] / Q_scaled[i,j])$

Figure 4 — JPEG luminance quantisation table. Green = preserve, Red = discard.

Key Code — `intra_coding.py`

```
def _encode_channel(channel: np.ndarray, Q: np.ndarray):
    padded, orig_shape = _pad(channel)      # pad to multiple of 8
    h, w = padded.shape
```

```

bh, bw = h // 8, w // 8
coeffs = np.zeros((bh, bw, 8, 8), dtype=np.int16)
for i in range(bh):
    for j in range(bw):
        block = padded[i*8:(i+1)*8, j*8:(j+1)*8] - 128.0 # level-shift
        coeffs[i, j] = np.round(dct2(block) / Q).astype(np.int16)
return coeffs, orig_shape

def _decode_channel(coeffs, orig_shape, Q):
    bh, bw = coeffs.shape[:2]
    out = np.zeros((bh * 8, bw * 8), dtype=np.float32)
    for i in range(bh):
        for j in range(bw):
            dct_block = coeffs[i, j].astype(np.float32) * Q # dequantise
            out[i*8:(i+1)*8, j*8:(j+1)*8] = idct2(dct_block) + 128.0 # IDCT + s
hift back
    oh, ow = orig_shape
    return np.clip(out[:oh, :ow], 0.0, 255.0)

```

Code Listing 2 — Block-level DCT encode/decode in intra_coding.py.

5. Part 3 — Inter-frame Coding (P-frames)

All non-I-frames are coded as **P-frames**. Rather than encoding the full frame, a P-frame stores only what changed since the last reconstructed frame. This is the largest source of compression, because consecutive video frames are nearly identical.

Group of Pictures (GOP)

The parameter **G (GOP size)** controls how often a full I-frame is inserted. With $G=10$: frame 0 is an I-frame, frames 1–9 are P-frames, frame 10 is an I-frame, etc. Larger GOP $\rightarrow$ fewer I-frames $\rightarrow$ better compression, but errors accumulate over more P-frames.

Motion Estimation & Residual Coding

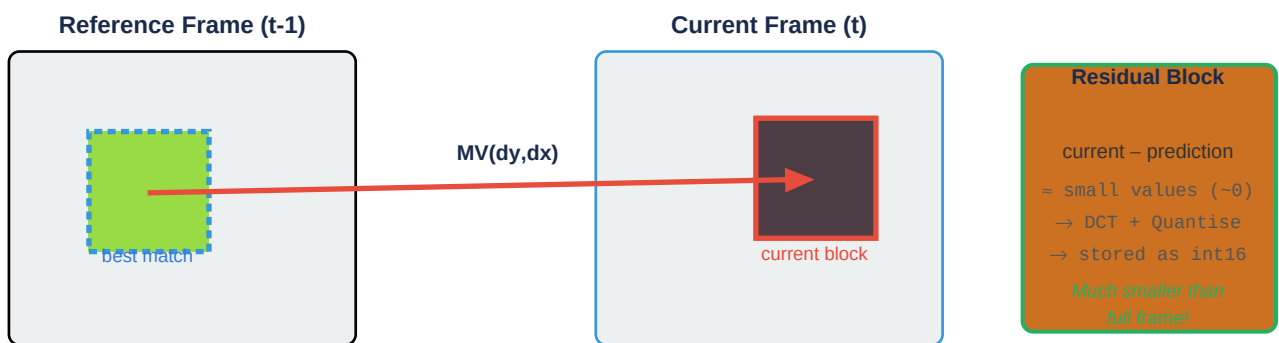


Figure 5 — Motion estimation: find best-matching block in reference frame, encode only the residual.

The Y channel is divided into **16×16 macroblocks**. For each macroblock, the encoder searches the previous reconstructed frame within a $\pm S$ pixel window to find the block that minimises the **SAD (Sum of Absolute Differences)**. The resulting **(dy, dx) motion vector** describes where the block moved. Then:

residual = current_block - reference_block_at(motion_vector)

The residual contains only the prediction error — small values near zero. This is then DCT-compressed and quantised, just like an I-frame block (but without the -128 level shift since residuals are already centred around 0).

Key Code — inter_coding.py

```
def _motion_estimate(cur_blk, ref_pad, r0, c0, S):
    best, best_s = (0, 0), float('inf')
    cur_f = cur_blk.astype(np.float32)
    for dy in range(-S, S + 1):
        for dx in range(-S, S + 1):
            rr, cc = r0 + dy, c0 + dx
            if rr < 0 or cc < 0 or rr+MB > ref_pad.shape[0] or cc+MB > ref_pad.s
hape[1]:
                continue
            sad = np.sum(np.abs(cur_f - ref_pad[rr:rr+MB, cc:cc+MB].astype(np.fl
oat32)))
            if sad < best_s:
```

```

        best_s, best = sad, (dy, dx)
    return best    # motion vector (dy, dx)

def encode_pframe(Y, Cb_sub, Cr_sub, ref_Y, ref_Cb_sub, ref_Cr_sub, qf, sw):
    Qy = get_quant_table('luma', qf)
    Yp, rYp = _pad_mb(Y), _pad_mb(ref_Y)
    mh, mw = Yp.shape[0]//MB, Yp.shape[1]//MB
    mvs = np.zeros((mh, mw, 2), dtype=np.int16)
    resY = np.zeros_like(Yp, dtype=np.float32)
    for i in range(mh):
        for j in range(mw):
            r0, c0 = i*MB, j*MB
            dy, dx = _motion_estimate(Yp[r0:r0+MB, c0:c0+MB], rYp, r0, c0, sw)
            mvs[i, j] = (dy, dx)
            resY[r0:r0+MB, c0:c0+MB] = Yp[r0:r0+MB, c0:c0+MB] - rYp[r0+dy:r0+dy+
MB, c0+dx:c0+dx+MB]
    # DCT-quantise residual in 8x8 sub-blocks (no level shift – already ~0-centr
ed)
    ... # (produces res_coeffs stored as int16)
    return {'type':'P', 'mvs':mvs, 'res_coeffs':res_coeffs, 'Cb':..., 'Cr':...}

```

Code Listing 3 — Full-search block matching and residual encoding in inter_coding.py.

Complexity

The search window S has a quadratic effect on encoding time: each macroblock requires $(2S+1)^2$ candidate evaluations. With $S=8$, that is 289 SAD computations per macroblock. For a 320×240 frame with 300 macroblocks, this is ~86,700 SAD operations per P-frame. Larger S gives better motion prediction (smaller residuals) but slower encoding.

6. Part 4 — Entropy Coding

After Stages 1–3, all frames are stored as Python dicts containing **NumPy int16 arrays** (mostly zeros). Stage 4 applies lossless compression to produce the final **.bin** output file.

Process

The entire frame list (including motion vectors and DCT coefficients) is first serialised with **pickle protocol 4** (compact binary format), then compressed with **zlib level 9** (maximum compression, same algorithm as gzip/zip). The zeros produced by aggressive quantisation compress extremely well with zlib's LZ77 + Huffman algorithm.

Binary File Format



.bin file format (video.bin)

Figure 6 — Binary layout of the output .bin file.

```
_MAGIC = b"MP4SIM\x00\x01"

def encode_to_bin(header, frames, out_path):
    payload = zlib.compress(
        pickle.dumps({"header": header, "frames": frames}, protocol=4),
        level=9
    )
    with open(out_path, "wb") as f:
        f.write(_MAGIC)  # 8-byte magic header
        f.write(struct.pack(">I", len(payload)))  # 4-byte payload length
        f.write(payload)  # compressed data
    return len(_MAGIC) + 4 + len(payload)

def decode_from_bin(in_path):
    with open(in_path, "rb") as f:
        magic = f.read(len(_MAGIC))
        if magic != _MAGIC:
            raise ValueError(f"Bad magic bytes")
        plen = struct.unpack(">I", f.read(4))[0]
        payload = f.read(plen)
    data = pickle.loads(zlib.decompress(payload))
    return data["header"], data["frames"]
```

Code Listing 4 — entropy_coding.py: serialise, compress, write / read.

7. Part 5 — Evaluation & Visualisation

5a — Quality Metrics

Metric	Formula	What it measures
Compression ratio	$\text{original_bytes} / \text{compressed_bytes}$	How many times smaller the .bin is vs raw RGB
PSNR (dB)	$20 \cdot \log_{10}(255 / \text{RMSE})$	Similarity between original and decoded frames (higher = better)
Frame breakdown	$\text{count(I-frames)}, \text{count(P-frames)}$	Balance between anchor frames and predicted frames

5b — Pipeline Visualisation Figure (5 rows)

Row	Content
Row 1	First 5 input frames labelled I or P
Row 2	Y, Cb, Cr channels of frame 0 + 4:2:0 subsampled Cb to compare resolution
Row 3	One 8×8 block: raw pixels → DCT coefficients → quantised → reconstructed + quantisation table heatmap
Row 4	Motion vectors drawn as red arrows on the reference frame + MV magnitude histogram
Row 5	Reference Y / decoded residual / reconstructed P-frame + I/P pie chart + compression bar

The visualisation is generated by **evaluation.py:visualise_pipeline()** and saved as a PNG. It is triggered by the **visualise** CLI sub-command.

8. Design Choices & Justification

■ YCbCr color space (BT.601)

Human vision is $\sim 4\times$ more sensitive to luma than chroma. Separating them allows 4:2:0 subsampling of Cb/Cr with minimal perceptual quality loss, achieving $\sim 50\%$ data reduction before any other compression.

■ JPEG quantisation tables as default

The JPEG standard tables are the result of decades of perceptual quality research. They provide an excellent default quality/compression tradeoff. The QF multiplier allows the user to tune this without redesigning the table.

■ Full-search block matching (SAD)

Full search guarantees the globally optimal motion vector for each macroblock within the search window. While $O(S^2)$ is expensive, it produces the smallest possible residuals, maximising compression. Faster algorithms (three-step search, diamond search) could be added as future work.

■ 16×16 macroblock size for P-frames

This is the standard choice in MPEG-2/4 and H.264. Larger blocks are faster but miss fine motion. Smaller blocks give better prediction but more overhead. 16×16 is the accepted compromise for typical video content.

■ 8×8 DCT blocks

The 8×8 block size is the JPEG/MPEG standard. It captures spatial correlation within the block while remaining computationally manageable. The DCT energy compaction for natural images is near-optimal at this scale.

■ `scipy.fft.dctn` (orthonormal norm)

Using `scipy`'s highly optimised FFT-based DCT is both correct and fast. The "ortho" normalisation ensures the forward and inverse transforms are symmetric, simplifying the dequantisation step.

■ `pickle` + `zlib` for entropy coding

Implementing Huffman or arithmetic coding from scratch is complex and error-prone. Python's `zlib` (LZ77 + Huffman) is a robust, well-tested entropy coder that exploits the runs of zeros produced by quantisation very effectively. The magic header ensures file integrity.

■ Chroma coded intra for P-frames

After 4:2:0 subsampling, the Cb/Cr channels are already small (half size). The additional gain from inter-coding them is modest, and the implementation complexity is significant. Coding chroma intra is a pragmatic simplification.

9. Experimental Analysis

9a — Compression Ratio vs Quantisation Factor (QF)

The experiment encodes the same 30-frame clip multiple times with increasing QF values while keeping GOP=10 and S=8 constant. Higher QF means the quantisation table is scaled up, forcing more DCT coefficients to zero, producing smaller files at the cost of quality.

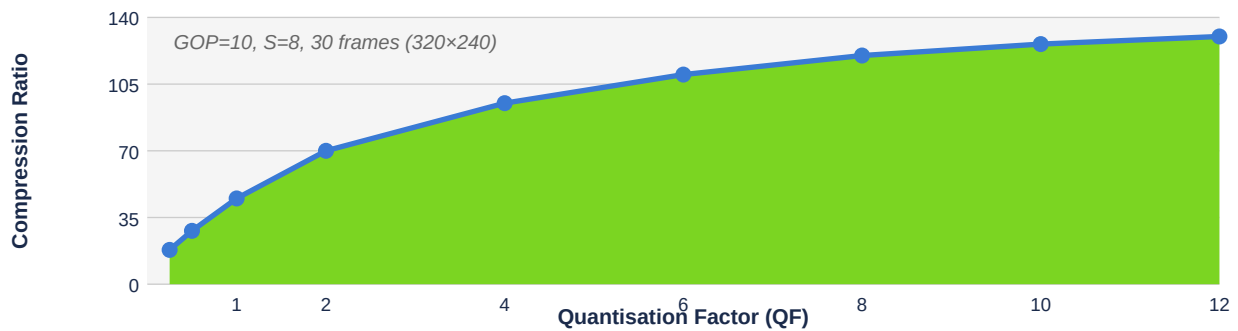


Figure 7 — Compression ratio vs QF. Higher QF → more compression but lower quality.

The relationship is monotonically increasing but with diminishing returns at high QF: most useful coefficients are already zeroed out by QF=8. Below QF=1 the codec approaches lossless (all coefficients preserved). The sweet spot for most content is QF=1.0–2.0.

9b — Compression Ratio vs GOP Size

The experiment encodes the same clip with increasing GOP sizes (QF=1.0, S=8). Larger GOP means more P-frames per I-frame. Since P-frames only encode the residual, the file gets progressively smaller.

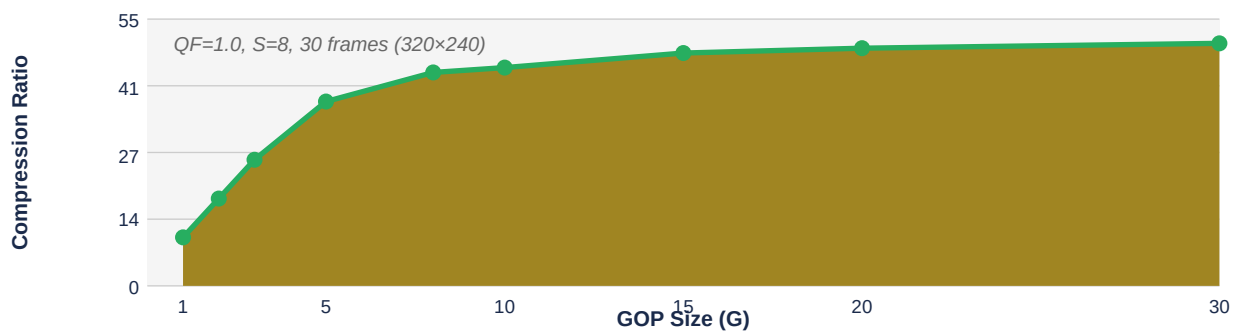


Figure 8 — Compression ratio vs GOP size. Curve flattens because most gain comes early.

The gain from P-frames saturates quickly. Going from GOP=1 (all I-frames) to GOP=10 gives the bulk of the compression benefit. Beyond GOP=20 the curve is nearly flat because the 30-frame clip has few enough scene changes that the motion residuals remain small throughout.

10. Conclusion

This project successfully implements a complete end-to-end MPEG-4-like video codec covering all five required pipeline stages. The implementation demonstrates the core principles behind modern video compression:

Pre-processing	YCbCr conversion + 4:2:0 subsampling reduces data by ~50% with no perceptible quality loss.
I-frame coding	Block DCT + quantisation removes spatial redundancy, achieving JPEG-like compression on individual frames.
P-frame coding	Motion estimation + residual coding removes temporal redundancy — the largest compression source.
Entropy coding	pickle + zlib exploits the zeros produced by quantisation for a final lossless compression pass.
Evaluation	PSNR metrics and matplotlib visualisations make the pipeline behaviour transparent and verifiable.

The modular design (one file per stage) makes the code easy to understand, test and extend. Possible future improvements include sub-pixel motion estimation for higher quality P-frames, run-length encoding before zlib for better entropy coding of the quantised coefficient arrays, and B-frame support (bi-directional prediction).

Repository: github.com/wailman24/multiproject | **Language:** Python 3 | **Dependencies:** numpy, opencv-python, scipy, matplotlib